Contents

Chapter 4

Interpolation and Least Squares

Interpolation is the process of defining a function that takes on specified values at specified points. Interpolation is used widely in image processing, in geographic information systems, market research, signal processing, etc.

Least squares approximation is when we have more data points then the degree of the polynomial approximation. Least squares approximation is used widely in statistics, data processing, etc.

4.1 Lagrange Interpolation

4.1.1 Lagrange form

Problem 4.1.1 *Given $n+1$ accurate data points*

$$(x_j, y_j) \quad \textit{for } j = 1, 2, \ldots, n+1,$$

find a polynomial P that interpolates the data, i.e.,

$$P(x_j) = y_j \quad \textit{for } j = 1, 2, \ldots, n+1.$$

Remark. No derivatives involved!

Example 4.1.1 Find a cubic polynomial P such that

$$P(0) = 2, \quad P(1) = 0, \quad P(2) = -3, \quad P(3) = 5.$$

Let

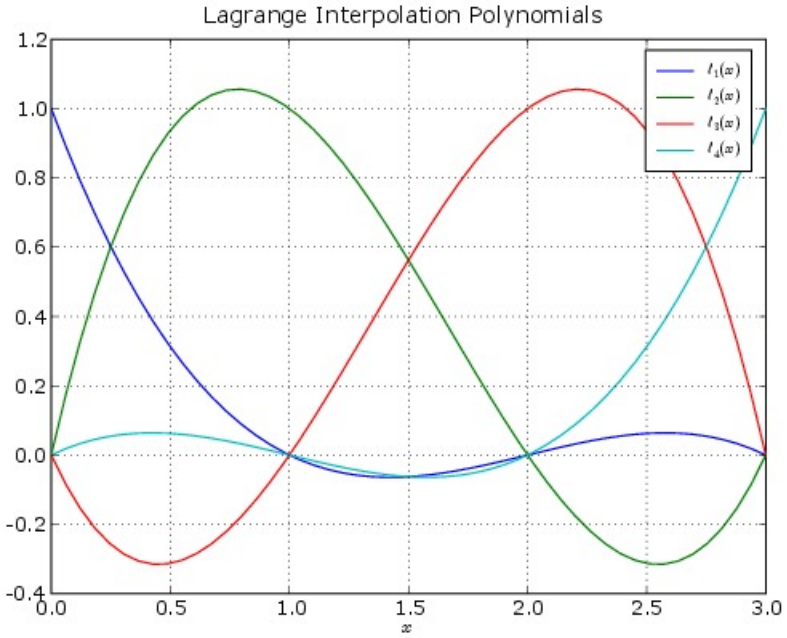
$$\ell_1(x) = \frac{(x-1)(x-2)(x-3)}{(0-1)(0-2)(0-3)} = \frac{-1}{6}(x-1)(x-2)(x-3),$$

so that

$$\ell_1(0) = 1 \quad \text{and} \quad \ell_1(1) = \ell_1(2) = \ell_1(3) = 0.$$

Similarly, we construct ℓ_2, ℓ_3, ℓ_4 satisfying

$$\begin{array}{lll} \ell_2(1) = 1 & \text{and} & \ell_2(0) = \ell_2(2) = \ell_2(3) = 0, \\ \ell_3(2) = 1 & \text{and} & \ell_3(0) = \ell_3(1) = \ell_3(3) = 0, \\ \ell_4(3) = 1 & \text{and} & \ell_4(0) = \ell_4(1) = \ell_4(2) = 0. \end{array}$$



Explicitly,

$$\ell_2(x) = \frac{(x-0)(x-2)(x-3)}{(1-0)(1-2)(1-3)} = \frac{1}{2}\,x(x-2)(x-3),$$

$$\ell_3(x) = \frac{(x-0)(x-1)(x-3)}{(2-0)(2-1)(2-3)} = \frac{-1}{2}\,x(x-1)(x-3),$$

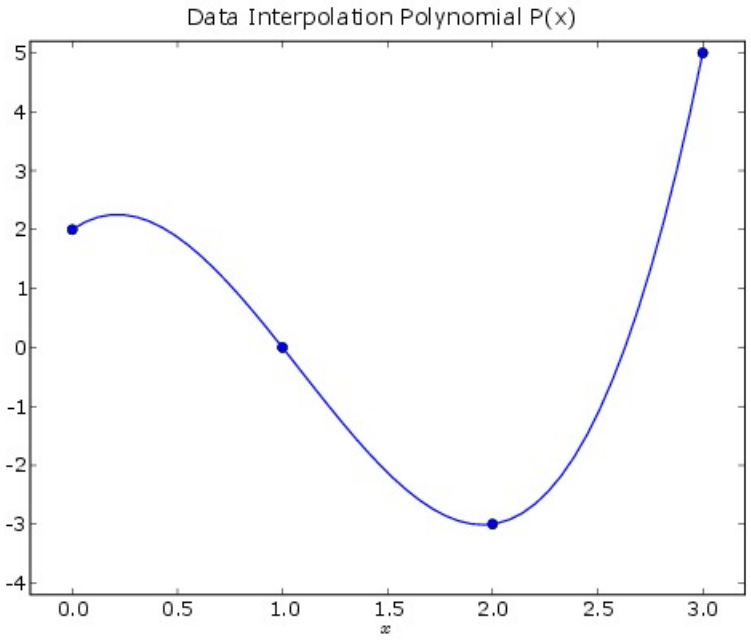
$$\ell_4(x) = \frac{(x-0)(x-1)(x-2)}{(3-0)(3-1)(3-2)} = \frac{1}{6}\,x(x-1)(x-2).$$

Now consider the cubic polynomial

$$P(x) = 2 \times \ell_1(x) + 0 \times \ell_2(x) - 3 \times \ell_3(x) + 5 \times \ell_4(x).$$

We have

$$\begin{aligned} P(0) &= 2 \times \ell_1(0) + 0 \times \ell_2(0) - 3 \times \ell_3(0) + 5 \times \ell_4(0) \\ &= 2 \times 1 \quad + 0 \times 0 \quad - 3 \times 0 \quad + 5 \times 0 = 2, \end{aligned}$$



$$\begin{aligned} P(1) &= 1 \times \ell_1(1) + 0 \times \ell_2(1) - 3 \times \ell_3(1) + 5 \times \ell_4(1) \\ &= 1 \times 0 \quad + 0 \times 1 \quad - 3 \times 0 \quad + 5 \times 0 = 0, \end{aligned}$$

and so on.

Definition 1 (Lagrange interpolation polynomials) *For general* $x_1 < x_2 < \cdots < x_{n+1}$ *we define the* Lagrange interpolation polynomials

$$\ell_j(x) = \prod_{\substack{k=1 \\ k \neq j}}^{n+1} \frac{x - x_k}{x_j - x_k}$$

so that

$$\ell_j(x_j) = 1 \quad \textit{and} \quad \ell_j(x_i) = 0 \quad \textit{for } i \neq j.$$

Hence, we obtain an interpolating polynomial of degree at most n by putting

$$P(x) = \sum_{j=1}^{n+1} y_j \ell_j(x).$$

Theorem 4.1.2 *Assuming that* $x_1 < x_2 < \cdots < x_n < x_{n+1}$*, there exists a unique polynomial* P *with degree* n *or less which interpolates the data* $(y_1, y_2, \ldots, y_n, y_{n+1})$.

Before the proof of the theorem is given, let us recall the *Fundamental theorem of algebra*:

Theorem 4.1.3 *If $Q(z)$ is a polynomial of degree n then there exist complex numbers $z_1, z_2, \ldots, z_n$ and a constant C such that $Q(z) = C(z - z_1)(z - z_2)\cdots(z - z_n)$*

Proof.
Existence: From the Lagrange form of interpolating polynomial.
Uniqueness: Let Q be another interpolating polynomial of degree n or less, then $P - Q$ has degree $m \leq n$ and satisfies

$$(P - Q)(x_j) = P(x_j) - Q(x_j) = y_j - y_j = 0 \quad \text{for } j = 1, 2, \ldots, n + 1.$$

Therefore, for some constant A,

$$(P - Q)(x) = A(x - x_1)(x - x_2)\cdots(x - x_m).$$

Since

$$0 = (P - Q)(x_{n+1}) = A(x_{n+1} - x_1)(x_{n+1} - x_2)\cdots(x_{n+1} - x_m)$$

we conclude that $A = 0$ and hence $P - Q \equiv 0$, implying that

$$P(x) = Q(x) \quad \text{for all } x.$$

□

The MATLAB function `polyfit` may be used to compute the interpolating polynomial for a set of data.

MATLAB commands

Example 4.1.2

```
>> xdata = [0:3];
>> ydata = [2, 0, -3, 5];
>> p = polyfit(xdata, ydata, 3);
>> x = linspace(0, 3);
>> y = polyval(p, x);
>> plot(x, y, xdata, ydata, 'o')
```

Python script

```
import numpy as np
import matplotlib.pyplot as plt

xdata = np.array([0,1,2,3])
ydata = np.array([2,0,-3,5])

P = np.polyfit(xdata,ydata,3)
# plot the results
x = np.linspace(0,3,200)
y = np.polyval(P,x)
plt.plot(x,y,'-',xdata,ydata,'ro',x,yp,'--')
plt.show()
```

Here, `polyfit` computes the coefficients of the interpolating polynomial, which we store in the vector `p`. The function `polyval` evaluates this polynomial at the points in the vector `x`.

4.1.2 Newton form

Suppose we wish to interpolate some function values

$$y_j = f(x_j) \quad \text{for } j = 1, 2, \ldots, n+1.$$

For $m = 0, 1, \ldots, n$, let P_m denote the interpolation polynomial of degree at most m that interpolates the first $m+1$ points, i.e.,

$$P_m(x_j) = f(x_j) \quad \text{for } j = 1, 2, \ldots, m+1.$$

Then

- $P_0(x) = f(x_1)$.
- $P_1 - P_0$ has degree at most 1 and satisfies

$$(P_1 - P_0)(x_1) = P_1(x_1) - P_0(x_1) = 0.$$

- Hence, there exists $a_1 \in \mathbb{R}$ such that

$$(P_1 - P_0)(x) = a_1(x - x_1),$$

  i.e.,

$$P_1(x) = f(x_1) + a_1(x - x_1).$$

Repeating the argument, we see that

$$\begin{aligned} P_0(x) &= f(x_1), \\ P_1(x) &= f(x_1) + a_1(x - x_1), \\ P_2(x) &= f(x_1) + a_1(x - x_1) + a_2(x - x_1)(x - x_2). \end{aligned}$$

and so on.

The *Newton form* of the interpolating polynomial P_n is

$$P_n(x) = f(x_1) + a_1(x - x_1) + \cdots + a_n(x - x_1)(x - x_2)\cdots(x - x_n).$$

Definition 2 (Divided difference) *The coefficients a_1, a_2, ..., a_n are called the* divided differences *of f with respect to the points x_j, and are usually denoted by*

$$a_j = f[x_1, x_2, \ldots, x_{j+1}], \quad j = 1, \ldots, n.$$

By expanding all terms in the Newton form, we see that

$$f[x_1, x_2, \ldots, x_j, x_{j+1}] = \text{coefficient of } x^j \text{ in } P_j(x).$$

To computing the divided differences $f[x_j]$ and $f[x_j, x_{j+1}]$, we define

$$f[x_1] = f(x_1), \quad f[x_2] = f(x_2), \quad \ldots \quad f[x_{n+1}] = f(x_{n+1}).$$

Since

$$a_1 = \frac{P_1(x) - P_0(x)}{x - x_1},$$

we have with $x = x_2$

$$a_1 = \frac{f(x_2) - f(x_1)}{x_2 - x_1} \quad \text{or} \quad f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1}$$

In general, by considering two points x_j and x_{j+1} we have

$$f[x_j, x_{j+1}] = \frac{f[x_{j+1}] - f[x_j]}{x_{j+1} - x_j}, \quad j = 1, \ldots, n.$$

In the following, we will show how to compute the divided differences $f[x_j, x_{j+1}, x_{j+2}]$
We have

$$P_1(x) = f[x_1] + f[x_1, x_2](x - x_1), \tag{4.1.1}$$
$$P_2(x) = P_1(x) + f[x_1, x_2, x_3](x - x_1)(x - x_2). \tag{4.1.2}$$

Now if Q_1 is the interpolating polynomial of degree 1 that interpolates f at x_2 and x_3, then

$$Q_1(x) = f[x_2] + f[x_2, x_3](x - x_2), \tag{4.1.3}$$
$$P_2(x) = Q_1(x) + f[x_2, x_3, x_1](x - x_2)(x - x_3). \tag{4.1.4}$$

Since $f[x_1, x_2, x_3]$ and $f[x_2, x_3, x_1]$ are the coefficient of x^2 in $P_2(x)$, they are equal. Note that the divided differences do not depend on the ordering of the points x_j.

It follows from (4.1.1) and (4.1.3) that

$$P_1(x) - Q_1(x) = (f[x_1, x_2] - f[x_2, x_3])x + \text{constant}. \tag{4.1.5}$$

It follows from (4.1.2) and (4.1.4) that

$$P_1(x) - Q_1(x) = f[x_1, x_2, x_3](x_1 - x_3)x + \text{constant}. \tag{4.1.6}$$

By equating the coefficient of x in (4.1.5) and (4.1.6) we derive

$$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}.$$

In general, by considering three points x_j, x_{j+1} and x_{j+2} we have

$$f[x_j, x_{j+1}, x_{j+2}] = \frac{f[x_{j+1}, x_{j+2}] - f[x_j, x_{j+1}]}{x_{j+2} - x_j}.$$

Using the same method we can prove

$$f[x_1, x_2, x_3, x_4] = \frac{f[x_2, x_3, x_4] - f[x_1, x_2, x_3]}{x_4 - x_1}.$$

By induction we have

$$f[x_1, x_2, \ldots, x_{k+1}] = \frac{f[x_2, x_3, \ldots, x_{k+1}] - f[x_1, x_2, \ldots, x_k]}{x_{k+1} - x_1}.$$

Using the formulae above, we may build up the *divided difference table* from a table of values of f:

$$\begin{array}{llllll}
x_1 & f(x_1) & & & & \\
 & & f[x_1, x_2] & & & \\
x_2 & f(x_2) & & f[x_1, x_2, x_3] & & \\
 & & f[x_2, x_3] & & f[x_1, x_2, x_3, x_4] & \\
x_3 & f(x_3) & & f[x_2, x_3, x_4] & & \\
 & & f[x_3, x_4] & & & \\
x_4 & f(x_4) & & & &
\end{array}$$

The coefficients of the Newton interpolating polynomial appear as the first entries in the columns.

Example 4.1.3 The divided difference table for our earlier data is

$$\begin{array}{cccccc} x & y & & & & \\ & & -2 & & & \\ 0 & 2 & & -\frac{1}{2} & & \\ & & -3 & & 2 & \\ 1 & 0 & & \frac{11}{2} & & \\ & & 8 & & & \\ 2 & -3 & & & & \\ & & & & & \\ 3 & 5 & & & & \end{array}$$

giving the Newton interpolation polynomial

$$P_3(x) = 2 - 2x - \tfrac{1}{2}x(x-1) + 2x(x-1)(x-2).$$

The MATLAB function `dd` computes the divided difference table `t`.

MATLAB function

```
function t = dd(xdata, ydata)
n = length(xdata);
t = zeros(n,n+1);
t(:,1) = xdata(:);
t(:,2) = ydata(:);
for j = 3:n+1
  i = 1:n-j+2;
  t(i,j) = ( t(i+1,j-1) - t(i,j-1) ) ./ ...
           ( t(i+j-2,1) - t(i,1) );
end
```

For instance:

MATLAB

```
>> xdata=[0:3]; ydata=[2,0,-3,5];
>> dd(xdata,ydata)

         0    2.0000   -2.0000   -0.5000    2.0000
    1.0000         0   -3.0000    5.5000         0
    2.0000   -3.0000    8.0000         0         0
    3.0000    5.0000         0         0         0
```

Python function

```
def ddtab(xdata,ydata):
# t = dd(xdata,ydata) computes the divided difference table.
# E.g., if xdata = [x1, x2, x3] and ydata = [f(x1), f(x2), f(x3)] then
#
#                  x1  f[x1]  f[x1,x2]  f[x1,x2]
#             t =  x2  f[x2]  f[x2,x3]  0
#                  x3  f[x3]  0         0
#
#
  import numpy as np
  n = len(xdata)
  if (len(ydata) != n):
      print('xdata and ydata must be the same length')
      return
  t = np.zeros((n,n+1),dtype=float)
  for k in range(0,n):
     t[k,0] = xdata[k]
     t[k,1] = ydata[k]
  for j in range(2,n+1):
     for i in range(0,n-j+1):
        t[i,j] = ( t[i+1,j-1] - t[i,j-1] )/ (t[i+j-1,0] -t[i,0])
        #print(i,j,t[i,j],'\n')
  return t
  # j=2
  #   t[i,2]= (t[i+1,1] - t[i,1])/ (t[i+1,0]-t[i,0])
```

Python script

```
import dd
xdata = [0,1,2,3]
ydata = [2,0,-3,5]
tab = dd.ddtab(xdata,ydata)
print(tab)
```

Python output

```
[[ 0.   2.  -2.  -0.5  2. ]
 [ 1.   0.  -3.   5.5  0. ]
 [ 2.  -3.   8.   0.   0. ]
 [ 3.   5.   0.   0.   0. ]]
```

Example 4.1.4 Given the following tabulated values of the function $f(x)$, use linear (degree $n = 1$) and cubic (degree $n = 3$) interpolation to estimate $f(1.3)$.

x	$f(x)$
1.00	0.08825 69642
1.20	0.22808 35032
1.40	0.33789 51297
1.60	0.42042 68964

We find

$$f[1.2, 1.4] = \frac{f(1.4) - f(1.2)}{1.4 - 1.2} = \frac{0.10981\,16265}{0.2} = 0.54905\,81323$$

so the linear interpolant for the points 1.2 and 1.4 is

$$\begin{aligned} P_1(x) &= f(1.2) + f[1.2, 1.4](x - 1.2) \\ &= 0.22808\,35032 + 0.54905\,81323\,(x - 1.2). \end{aligned}$$

The interpolated value is

$$P_1(1.3) = 0.22808\,35032 + 0.54905\,81323 \times 0.1 = 0.28298\,93165.$$

To use cubic interpolation, we compute the divided difference table

$$\begin{array}{lllll} 1.0 & 0.0882569642 & & & \\ & & 0.6991326951 & & \\ 1.2 & 0.2280835032 & & -0.3751864070 & \\ & & 0.5490581323 & & 0.0569802676 \\ 1.4 & 0.3378951297 & & -0.3409982465 & \\ & & 0.4126588337 & & \\ 1.6 & 0.4204268964 & & & \end{array}$$

and form

$$\begin{aligned} P_3(x) = 0.0882569642 &+ 0.6991326951(x - 1.0) \\ &- 0.3751864070(x - 1.0)(x - 1.2) \\ &+ 0.0569802676(x - 1.0)(x - 1.2)(x - 1.4). \end{aligned}$$

The interpolated value is

$$\begin{aligned} P_3(1.3) &= 0.0882569642 + 0.6991326951 \times 0.3 \\ &\quad - 0.3751864070 \times 0.3 \times 0.1 \\ &\quad + 0.0569802676 \times 0.3 \times 0.1 \times -0.1 \\ &= 0.0882569642 + 0.2097398085 \\ &\quad - 0.0112555922 - 0.0001709408 \\ &= 0.2865702397. \end{aligned}$$

It turns out that in this example the exact value is

$$f(1.3) = 0.28653535716557,$$

so the interpolation error is

$$P_3(1.3) - f(1.3) = -3.49 \times 10^{-5}.$$

4.2 Hermite Interpolation

Problem 4.2.1 *Given n nodes x_1, x_2, ..., x_n, find a polynomial P that satisfies*

$$P^{(j)}(x_i) = c_{ij}, \quad 0 \leq j \leq k_i, \quad 1 \leq i \leq n. \tag{4.2.1}$$

The total number of conditions on P is

$$M = (k_1 + 1) + (k_2 + 1) + \cdots + (k_n + 1) = n + k_1 + \cdots + k_n.$$

Theorem 4.2.2 *There exists a unique polynomial P of degree $M - 1$ that satisfies* (4.2.1).

Example 4.2.1 Find a polynomial P that assumes these values

$$P(0) = 0, \quad P(1) = 1, \quad P'(1) = 2.$$

Example 4.2.2 What happens in Hermite interpolation when there is only one node?

We will not study this type of interpolation further.

4.3 The interpolation error

In this section, we will study the error of the Lagrange interpolation in details.

4.3.1 The error function

Let P_n be the interpolating polynomial that interpolates f at $x_1, \ldots, x_{n+1}$. Let

$$E_n(x) = f(x) - P_n(x),$$

so that $|E_n(x)|$ is the absolute error in the approximation

$$P_n(x) \approx f(x).$$

Obviously, this error is zero if $x = x_j$ and $1 \leq j \leq n + 1$.

Definition 3 *We say that f is C^r if f, f', f'', ..., $f^{(r)}$ exist and are continuous.*

Theorem 4.3.1 *If f is C^{n+1} on (x_1, x_{n+1}) then for each x with $x_1 < x < x_{n+1}$ there exists $\xi \in (x_1, x_{n+1})$ such that*

$$E_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_1)(x - x_2)\cdots(x - x_{n+1}).$$

Before the proof of the theorem is given, we state the mean value theorem for divided differences.

Theorem 4.3.2 *For any set of points $x_1 < x_2 < \ldots < x_{n+1}$, if f is C^n on (x_1, x_{n+1}) then there exists an interior point $\xi \in (x_1, x_{n+1})$ so that*

$$f[x_1, \ldots, x_{n+1}] = \frac{f^{(n)}(\xi)}{n!}.$$

Proof. Let P be the Lagrange interpolation polynomial for f at $x_1, \ldots, x_{n+1}$. Then it follows from the Newton form that the highest term of P is $f[x_1, x_2, \ldots, x_{n+1}]x^n$.

Let g be the remainder of the interpolation, defined by $g = f - P$. Then g has $n + 1$ zeros: $x_1, \ldots, x_{n+1}$. By applying Rolle's theorem first to g, then to g', and so on until $g^{(n-1)}$, we find that $g^{(n)}$ has a zero ξ. This means that

$$0 = g^{(n)}(\xi) = f^{(n)}(\xi) - f[x_1, \ldots, x_{n+1}]n!,$$

or equivalently

$$f[x_1, \ldots, x_{n+1}] = \frac{f^{(n)}(\xi)}{n!}.$$

□

Proof. [of theorem 4.3.1] The Newton form of the interpolation polynomial P_n at $x_1, \ldots, x_{n+1}$,

$$P_n(x) = f(x_1) + f[x_1, x_2](x - x_1) + \ldots + f[x_1, x_2, \ldots, x_{n+1}](x - x_1)\cdots(x - x_n).$$

Now let P_{n+1} be the interpolation polynomial at $x_1, \ldots, x_{n+1}, z$. The Newton's form for P_{n+1} is

$$\begin{aligned} P_{n+1}(x) &= f(x_1) + f[x_1, x_2](x - x_1) + \ldots + f[x_1, x_2, \ldots, x_{n+1}](x - x_1)\cdots(x - x_n) \\ &\quad + f[x_1, x_2, \ldots, x_{n+1}, z](x - x_1)\cdots(x - x_n)(x - x_{n+1}) \\ &= P_n(x) + f[x_1, x_2, \ldots, x_{n+1}, z](x - x_1)\cdots(x - x_n)(x - x_{n+1}). \end{aligned}$$

Hence,

$$P_{n+1}(x) - P_n(x) = f[x_1, x_2, \ldots, x_{n+1}, z](x - x_1)\cdots(x - x_n)(x - x_{n+1}).$$

However, at $x = z$, since P_{n+1} interpolates f, we have $f(z) = P_{n+1}(z)$ and hence we get the error expression

$$f(z) - P_n(z) = f[x_1, \ldots, x_{n+1}, z] \prod_{i=1}^{n+1} (z - x_i).$$

Since z is a dummy variable, we can re-label z as x and get the expression given in the statement of the theorem

$$f(x) - P_n(x) = f[x_1, \ldots, x_{n+1}, x] \prod_{i=1}^{n+1} (x - x_i).$$

If f in $C^{(n+1)}$ on the interval then by the mean value for divided differences,

$$f[x_1, \ldots, x_{n+1}, x] = \frac{f^{(n+1)}(\xi)}{(n+1)!}.$$

Hence the theorem is proved. □

Example 4.3.1 It is known that the function f satisfies

$$|f'''(x)| \leq 3 \quad \text{for } 0 \leq x \leq 2.$$

Use this bound to estimate the absolute error in the approximation

$$f(1.5) \approx P_2(1.5)$$

if $P_2(x)$ is the quadratic polynomial that interpolates $f(x)$ at the points $x_1 = 0$, $x_2 = 1$, $x_3 = 2$.

Answer. Using Theorem 4.3.1 we have

$$|f(x) - P_2(x)| \leq \frac{|f'''(\xi)|}{3!} |(x - x_1)(x - x_2)(x - x_3)|.$$

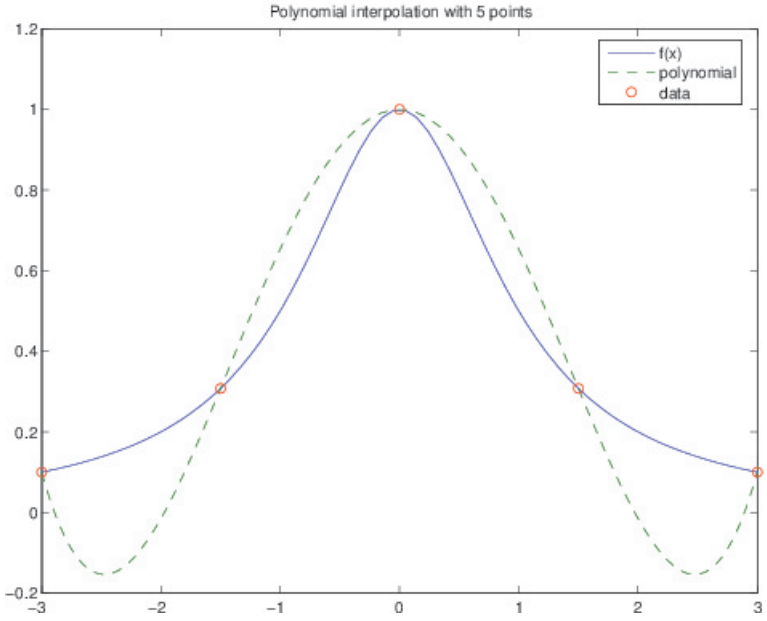
With $x = 1.5$ we have

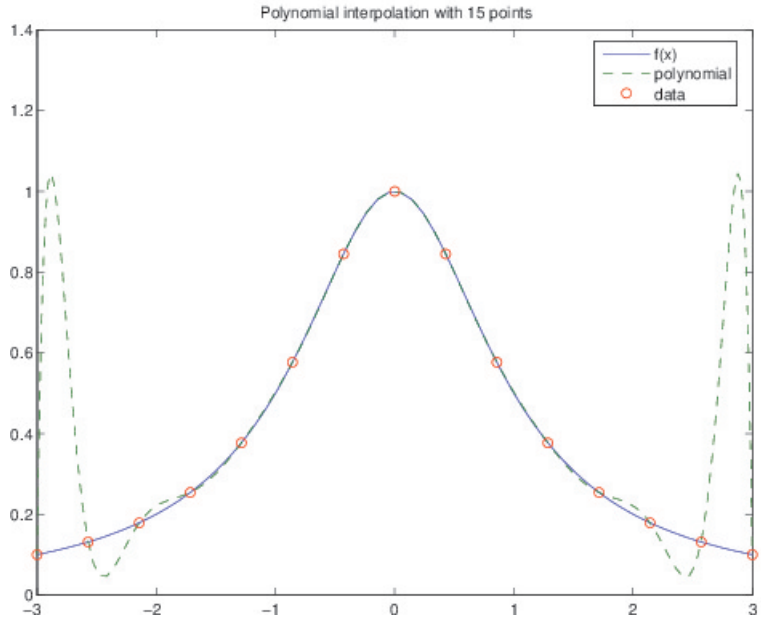
$$|f(1.5) - P_2(1.5)| \leq \frac{3}{6} |(1.5 - 0)(1.5 - 1)(1.5 - 2)| = 0.1875.$$

Example 4.3.2 Consider the function

$$f(x) = \frac{1}{1 + x^2} \quad \text{for } -3 \leq x \leq 3.$$

We interpolate at $n + 1$ equally-spaced points in the interval $[-3, 3]$:



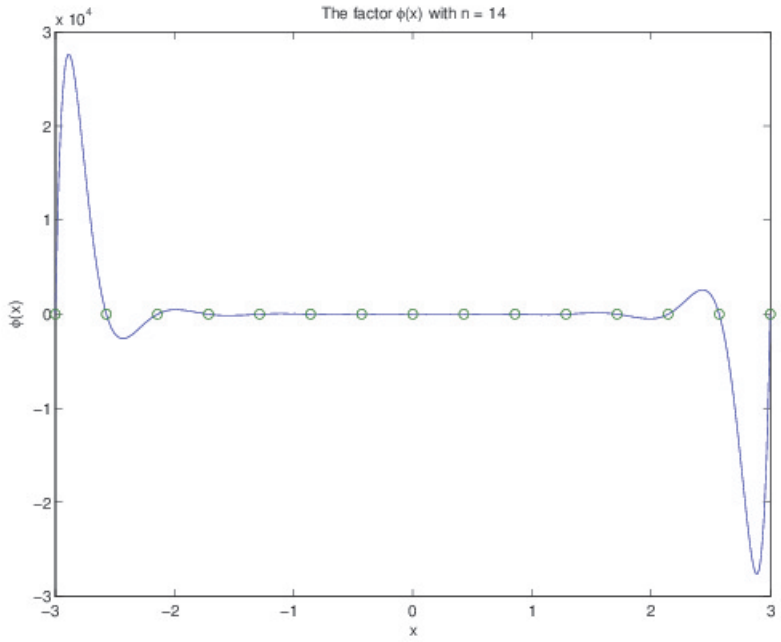


As we increase n the approximation $P_n(x) \approx f(x)$ improves rapidly for x near the middle of the interpolation interval $[-3, 3]$, but wild oscillations occur near the endpoints of the interval.

This behaviour is explained by the factor

$$\phi(x) = (x - x_1)(x - x_2) \cdots (x - x_{n+1})$$

appearing in $E_n(x)$.

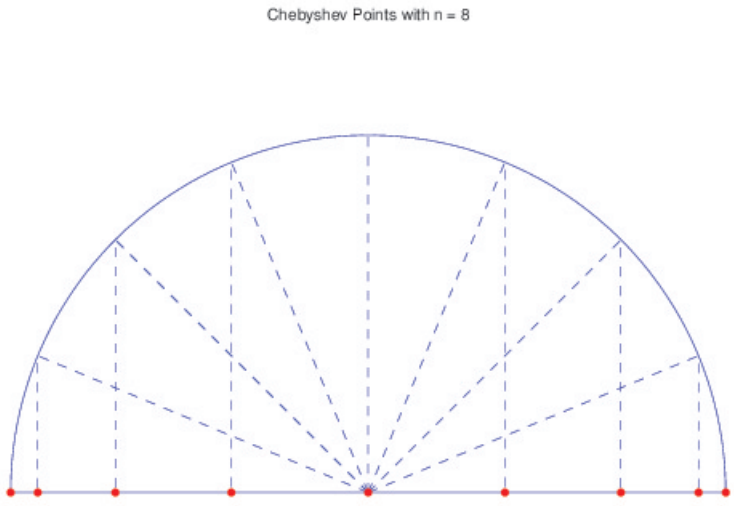


4.3.2 Chebyshev points

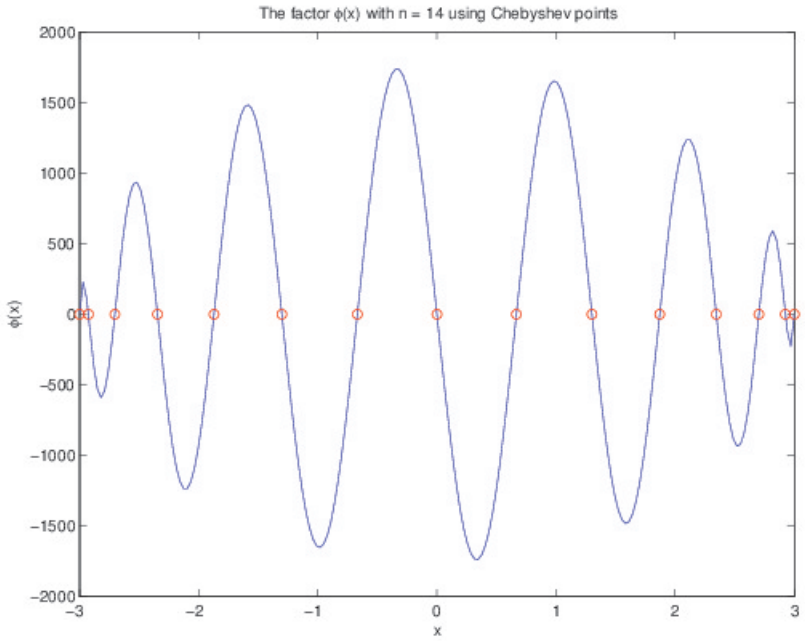
For smooth f, we can obtain a uniformly small interpolation error by bunching up the points near the ends of the interpolation interval. A good choice for the interval $[a, b] = [x_1, x_{n+1}]$ is given by the Chebyshev points

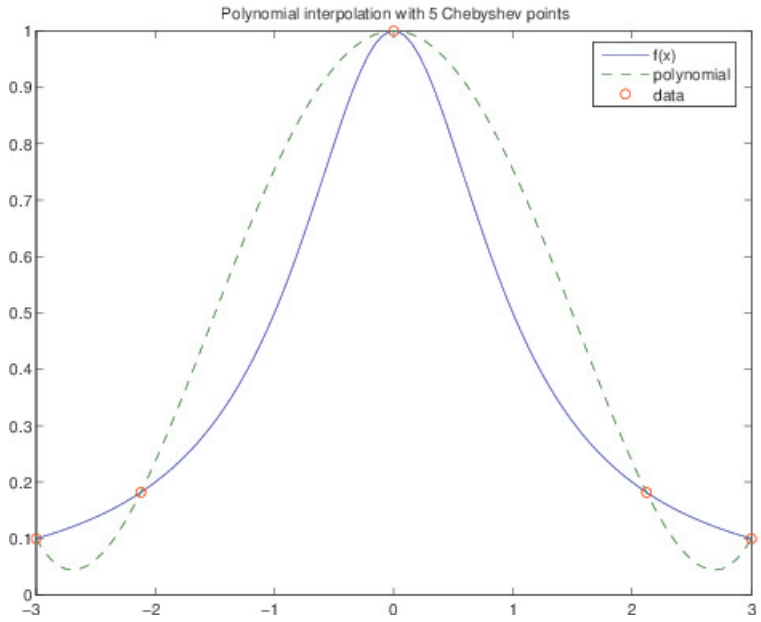
$$x_j = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{n+1-j}{n}\,\pi\right) \quad \text{for } j = 1, 2, \ldots, n+1.$$

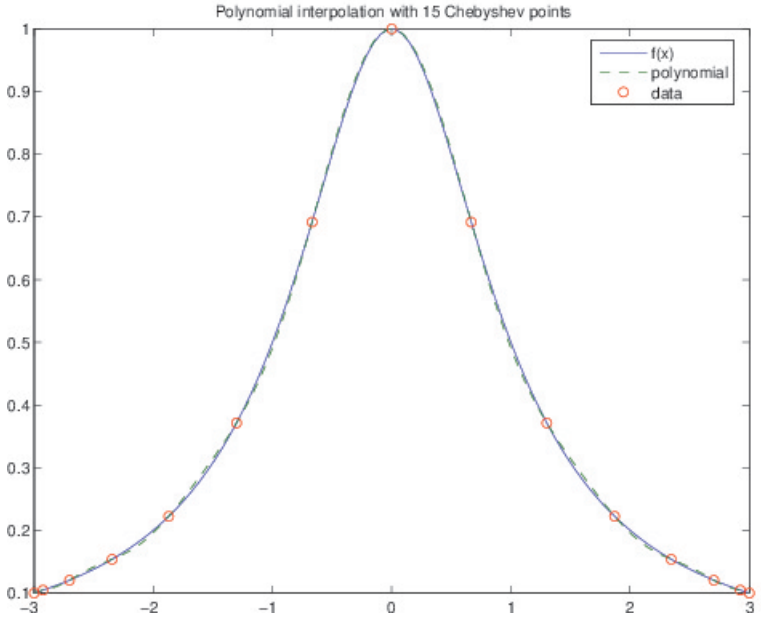
Geometrically, these x_j can be obtained by taking equally-spaced points on a semi-circle and projecting them onto the diameter.



Using Chebyshev points, the oscillations in $\phi(x)$ are spread more evenly over the interpolation interval and have a much smaller maximum amplitude. This has the expected effect on the interpolation error.







The following table shows the behaviour of the interpolation error

$$\max_{-3\le x\le 3} |f(x) - P_n(x)|$$

for the function $f(x) = 1/(1+x^2)$ considered earlier, using increasing numbers of Chebyshev points.

n	error	ratio
10	3.0722e-02	
20	1.2420e-03	0.0404
30	4.8084e-05	0.0387
40	1.8325e-06	0.0381
50	6.9532e-08	0.0379
60	2.6342e-09	0.0379
70	9.9716e-11	0.0379
80	3.7748e-12	0.0379
90	1.4311e-13	0.0379
100	6.6613e-15	0.0465

Example 4.3.3 Using MATLAB or Python, find the polynomial that interpolates $f = 1/(1+x^2)$ at $n+1$ equally-spaced points in $[-3, 3]$.

MATLAB script

```
n = 4
xdata = linspace(-3,3,n+1);
ydata = 1 ./ ( 1 + xdata.^2 );
x = linspace(-3,3,200);
y = 1 ./ ( 1 + x.^2 );
P = polyfit(xdata, ydata, n);
plot(x, y, x, polyval(P,x), '--', xdata, ydata, 'o')
title(['Polynomial interpolation with ' num2str(n+1) ' points'])
legend('f(x)', 'polynomial', 'data')
```

Python script

```
import numpy as np
import math
import matplotlib.pyplot as plt

n=4
xdata = np.linspace(-3,3,n+1)
ydata = 1/ (1+ xdata**2)
x = np.linspace(-3,3,200)
y = 1/ (1+x**2)
P = np.polyfit(xdata,ydata,n)
yp = np.polyval(P,x)
plt.plot(x,y,'-',xdata,ydata,'ro',x,yp,'--')
plt.show()
```

Example 4.3.4 The following MATLAB function evaluates $\phi(x)$ in Example 13.

```
function y = phi(x, xdata)
% y = phi(x, xdata) computes phi(x) =
% (x-xdata(1))(x-xdata(2))...(x-xdata(end))
% x and xdata must be row vectors.
x = x(:)';
xdata = xdata(:)';
M = repmat(x, length(xdata), 1) - ...
    repmat(xdata', 1, length(x));
y = prod(M);
```

Example 4.3.5 The MATLAB function `chebyspace` computes the Chebyshev points:

MATLAB function

```
function x = chebyspace(a, b, m)
  n = m - 1;
  x(1) = a;
  x(2:n) = 0.5*(a+b)+0.5*(b-a)*cos([n-1:-1:1]*pi/n);
  x(m) = b;
```

The following Python script will do the same task.

Python script

```
import numpy as np
def chebyspace(a,b,m):
   n = m-1
   x = np.zeros(m)
   x[0] = a
   x[m-1] = b
   for j in range(n-1,0,-1):
      x[n-j] = 0.5*(a+b) + 0.5*(b-a)*np.cos(j*np.pi/n)
   return x

# main function
a = -3
b = 3
m = 15
xx = chebyspace(a,b,m)
print(xx)
```

4.4 Least Squares Problems

4.4.1 Line of best fit

Example 4.4.1 Find the line $\ell(t) = \alpha + \beta t$ of best fit to the data (t_j, y_j) for $j = 1, \ldots, m$ (for $m > 2$).

Let the residual r_j be defined by

$$r_j = (\alpha + \beta t_j) - y_j, \quad j = 1, \ldots, m.$$

We would like to best fit the data by mimimizing the sum of squares of the residuals, that is

$$\text{miminize} \sum_{j=1}^{m} r_j^2.$$

Let the parameter vector $\boldsymbol{x}$, the coefficient matrix A, the data vector $\boldsymbol{y}$ be defined by

$$\boldsymbol{x} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad A = \begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix}, \quad \boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix},$$

Then we can write the residual vector $\boldsymbol{r} = [r_1, \ldots, r_m]^T$ as

$$\boldsymbol{r} = A\boldsymbol{x} - \boldsymbol{y} = (\alpha + \beta t_1 - y_1, \ldots, \alpha + \beta t_m - y_m)^T$$

The least squares problem is
Minimize $F = \|\boldsymbol{r}\|_2^2 = \boldsymbol{r}^T\boldsymbol{r} = \sum_{j=1}^m (\alpha + \beta t_j - y_j)^2$
Set $\partial F/\partial\alpha = 0$ and $\partial F/\partial\beta = 0$ we obtain

$$\begin{cases} \alpha m + \beta \sum_{j=1}^m t_j & = \sum_{j=1}^m y_j \\ \alpha \sum_{j=1}^m + \beta \sum_{j=1}^m t_j^2 & = \sum_{j=1}^m y_j t_j \end{cases}$$

Alternatively, write $F = \boldsymbol{r}^T\boldsymbol{r} = \boldsymbol{x}^T A^T A\boldsymbol{x} - 2\boldsymbol{x}^T A^T\boldsymbol{y} + \boldsymbol{y}^T\boldsymbol{y}$ and set $\partial F/\partial\alpha = \partial F/\partial\beta = 0$, we also obtain the normal equations

$$\left(A^T A\right)\boldsymbol{x} = A^T\boldsymbol{y},$$

which is a 2 by 2 linear system with

$$A^T A = \begin{bmatrix} m & \sum_{j=1}^m t_j \\ \sum_{j=1}^m t_j & \sum_{j=1}^m t_j^2 \end{bmatrix}, \quad A^T\boldsymbol{y} = \begin{bmatrix} \sum_{j=1}^m y_j \\ \sum_{j=1}^m t_j y_j \end{bmatrix}$$

Example 4.4.2 Use Matlab to find the line $y(t) = \alpha + \beta t$ of best fit to the data

j	1	2	3	4	5
t_j	0	0.5	1	1.5	2
y_j	1.0	0.6	0.4	0.1	0.08

In this case, $m = 5$ and $n = 2$.

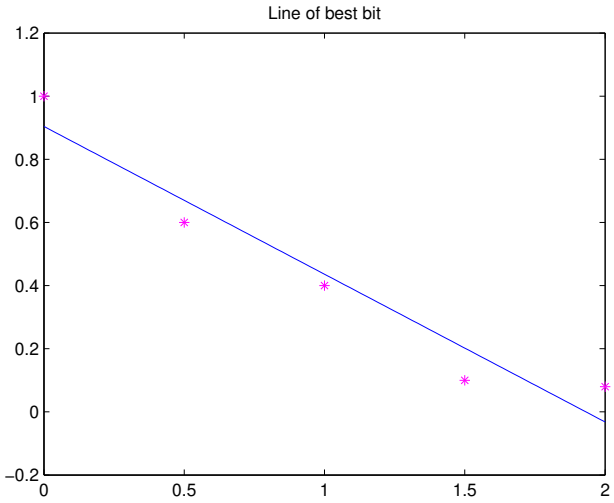
MATLAB script

```
tdat = [0:0.5:2];
ydat = [1.0 0.6 0.4 0.1 0.08];
tdat = tdat(:); % change to column vector
ydat = ydat(:);
A = [ones(size(tdat)) tdat];
x = A \ ydat
```

After running, we get $x(1) = 0.9040$ ans $x(2) = -0.4680$. The line of best fit is $y(t) = 0.904 - 0.468t$.

MATLAB script

```
t = linspace(0,2,100);
y = x(1) + x(2)*t;
plot(t,y,'b-',tdat,ydat,'m*');
```



Python script

```
import numpy as np

tdat = np.array([0,0.5,1.0,1.5,2.0])
ydat = np.array([1.0, 0.6, 0.4, 0.1, 0.08])
# assemble matrix A
A = np.vstack([np.ones(len(tdat)), tdat]).T

# turn ydat into a column vector
ydat = ydat[:,np.newaxis]

# Solving the normal equation
B = np.dot(A.T,A)
rhs = np.dot(A.T,ydat)
x = np.linalg.solve(B,rhs)

# print the coefficients of the best fit line
print('x=',x)

# plotting the result
t = np.linspace(0,2,100)
y = x[0] + x[1]*t
plt.plot(t,y,'b-',tdat,ydat,'m*')
plt.show()
```

4.4.2 Quadratic of best fit

Example 4.4.3 Use Matlab to find the quadratic $y(t) = \alpha + \beta t + \gamma t^2$ of best fit to the data in the previous example.

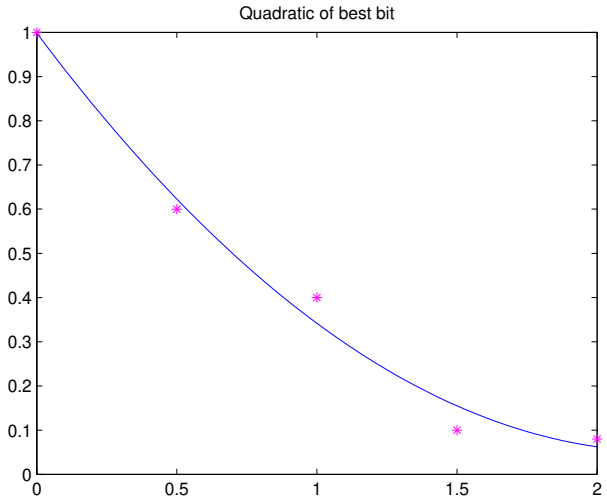
In this case, $m = 5$ and $n = 3$. The residual is

$$\boldsymbol{r} = (\alpha + \beta t_1 + \gamma t_1^2 - y_1, \ldots, \alpha + \beta t_m + \gamma t_m^2 - y_m)^T.$$

We can write $\boldsymbol{r} = A\boldsymbol{x} - \boldsymbol{y}$ where the coefficient matrix A is

$$\begin{bmatrix} 1 & t_1 & t_1^2 \\ 1 & t_2 & t_2^2 \\ \vdots & \vdots & \vdots \\ 1 & t_m & t_m^2 \end{bmatrix}$$

Define `tdat` and `ydat` as before. Then



MATLAB script

```
A = [ones(size(tdat)) tdat  tdat.^2];
x = A \ ydat
```

We get `x` $= [0.9983; -0.8451; 0.1886]$. Thus the quadratic of best fit is $y(t) = 0.9983 - 0.8451t + 0.1886t^2$.

MATLAB script

```
t = linspace(0,2,100);
y = x(1) + x(2)*t + x(3)*t.^2;
plot(t,y,'b-',tdat,ydat,'m*');
```

Python script

```
import numpy as np
import matplotlib.pyplot as plt

tdat = np.array([0,0.5,1.0,1.5,2.0])
ydat = np.array([1.0, 0.6, 0.4, 0.1, 0.08])
# assemble matrix A
A = np.vstack([np.ones(len(tdat)), tdat, tdat**2]).T

# turn ydat into a column vector
ydat = ydat[:,np.newaxis]

# Solving the normal equation
B = np.dot(A.T,A)
rhs = np.dot(A.T,ydat)
x = np.linalg.solve(B,rhs)

print('x=',x)

# plotting the result
t = np.linspace(0,2,100)
y = x[0] + x[1]*t + x[2]*t**2
plt.plot(t,y,'b-',tdat,ydat,'m*')
plt.show()
```

4.4.3 Best fit by a polynomial

Suppose we want to fit the data (t_j, y_j) for $j = 1, \ldots, m$ using a polynomial of degree $n < m$,

$$P_n(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \ldots + \alpha_n t^n$$

The residual vector is

$$\boldsymbol{r} = \begin{bmatrix} \alpha_0 + \alpha_1 t_1 + \ldots + \alpha_n t_1^n - y_1 \\ \alpha_0 + \alpha_1 t_2 + \ldots + \alpha_n t_2^n - y_2 \\ \vdots \quad \vdots \quad \vdots \quad \vdots \\ \alpha_0 + \alpha_1 t_n + \ldots + \alpha_n t_m^n - y_m \end{bmatrix}$$

As before, we can write

$$\boldsymbol{r} = A\boldsymbol{x} - \boldsymbol{y},$$

where $A \in \mathbb{R}^{m\times n}$, $m > n$, which is over determined (more equations than variables), and $\boldsymbol{x}$ and $\boldsymbol{y}$ are given by

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & \dots & t_1^n \\ 1 & t_2 & t_2^2 & \dots & t_2^n \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & t_m & t_m^2 & \dots & t_m^n \end{bmatrix} \quad \boldsymbol{x} = \begin{bmatrix} \alpha_0 \\ \vdots \alpha_n \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} y_1 \\ \vdots y_m \end{bmatrix}.$$

Since there are more equations than variables then do not expect a solution to exist.

We want to minimize norm of residual $\|\boldsymbol{r}\|$ via the least squares problem:

Minimize $\|\boldsymbol{r}\|_2^2 = \boldsymbol{r}^T\boldsymbol{r} = \sum_{j=1}^m r_j^2$

We have

$$\begin{aligned} \|\boldsymbol{r}\|^2 &= \|A\boldsymbol{x} - \boldsymbol{y}\|^2 = (A\boldsymbol{x} - \boldsymbol{y})^T(A\boldsymbol{x} - \boldsymbol{y}) \\ &= \boldsymbol{x}^T A^T A\boldsymbol{x} - \boldsymbol{x}^T A^T \boldsymbol{y} - \boldsymbol{y}^T A^T \boldsymbol{x} + \boldsymbol{y}^T \boldsymbol{y} \end{aligned}$$

Taking partial derivatives with respect to $\boldsymbol{x} = [\alpha_0, \dots, \alpha_n]^T$,

$$\begin{aligned} \frac{\partial \|\boldsymbol{r}\|^2}{\partial \boldsymbol{x}} &= \left[\frac{\partial \|\boldsymbol{r}\|^2}{\partial \alpha_0} \cdots \frac{\partial \|\boldsymbol{r}\|^2}{\partial \alpha_n}\right]^T \\ &= \frac{\partial}{\partial \boldsymbol{x}}(\boldsymbol{x}^T A^T A\boldsymbol{x} - \boldsymbol{x}^T A^T \boldsymbol{y} - \boldsymbol{y}^T A^T \boldsymbol{x} + \boldsymbol{y}^T \boldsymbol{y}) \\ &= 2A^T A\boldsymbol{x} - 2A^T \boldsymbol{y} \end{aligned}$$

Setting the gradient to zero, we obtain the normal equations

$$A^T A\boldsymbol{x} = A^T \boldsymbol{y} \Leftrightarrow \boldsymbol{x} = (A^T A)^{-1} A^T \boldsymbol{y}.$$

In practice solve we $\left(A^T A\right)\boldsymbol{x} = A^T\boldsymbol{y}$. The matrix $A^T A$ is symmetric positive definite coefficient matrix, hence Cholesky factorization can be used.

The remaining issue here is $\kappa_2(A^T A) = \kappa_2(A)^2$, squaring condition number of the coefficient matrix A.

4.4.4 Using exponential functions

Example 4.4.4 Approximate the data from the previous examples by an exponential function $y(t) = \lambda e^{\mu t}$.

Convert to a linear problem

$$y(t) = \lambda e^{\mu t} \text{ then } \ln y(t) = \ln(\lambda e^{\mu t}) = \ln \lambda + \mu t.$$

The data values yield a system of equations in λ and μ

$$\begin{cases} \ln\lambda + \mu t_1 & = \ln y_1 \\ \ln\lambda + \mu t_2 & = \ln y_2 \\ \vdots & \vdots \\ \ln\lambda + \mu t_m & = \ln y_m \end{cases}$$

The coefficient matrix

$$A = \begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} \ln\lambda \\ \mu \end{bmatrix} \quad \boldsymbol{b} = \begin{bmatrix} \ln y_1 \\ \ln y_2 \\ \vdots \\ \ln y_m \end{bmatrix}$$

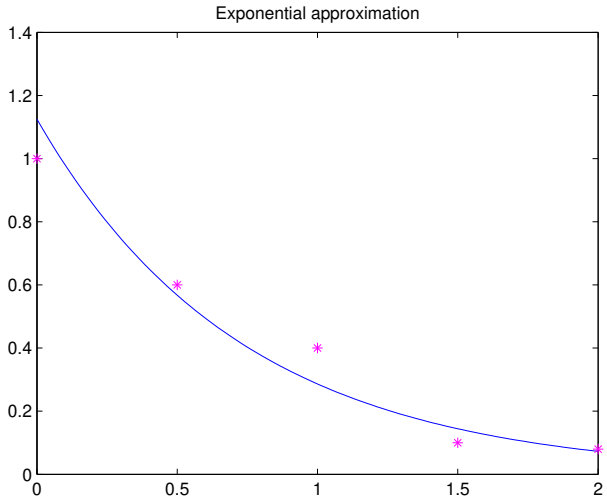
MATLAB script

```
A = [ones(size(tdat)) tdat];
x = A \ log(ydat)
lam = exp(x(1))
mu = x(2)
```

`lam = 1.1247` and `mu = -1.3686`. The exponential approximation is $y(t) = 1.1247e^{-1.3686t}$. Plotting

MATLAB script

```
t = linspace(0,2,100);
y = lam * exp(mu*t);
plot(t,y,'b-',tdat,ydat,'m*');
```



Python script

```
import numpy as np
import matplotlib.pyplot as plt

tdat = np.array([0,0.5,1.0,1.5,2.0])
ydat = np.array([1.0, 0.6, 0.4, 0.1, 0.08])
# assemble matrix A
A = np.vstack([np.ones(len(tdat)), tdat]).T

# turn ydat into a column vector
ydat = ydat[:,np.newaxis]

# Solving the normal equation
B = np.dot(A.T,A)
rhs = np.dot(A.T,np.log(ydat))
x = np.linalg.solve(B,rhs)

lam = np.exp(x[0])
mu = x[1]

# plotting the result
t = np.linspace(0,2,100)
y = lam*np.exp(mu*t)
plt.plot(t,y,'b-',tdat,ydat,'m*')
plt.show()
```